

The Java Development Kit

Java

Firstly they developed Greentalk language.

Java is a programming language created by James Gosling from Sun Microsystems (Sun) in 1991.

The target of Java is to write a program once and then run this program on multiple operating systems.

The first publicly available version of Java (Java 1.0) was released in 1995. Sun Microsystems was acquired by the Oracle Corporation in 2010.

Java is used for:

- Mobile applications (specially Android apps)
- Desktop applications
- Web applications
- Web servers and application servers
- Games
- Database connection
- And much, much more!

Why Use Java?

- Java is an object oriented language which gives a clear structure to programs and allows code to be reused, lowering development costs
- Java works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc.)
- It is one of the most popular programming language in the world.
- It is easy to learn and simple to use
- It is open-source and free
- It is secure, fast and powerful
- It has a huge community support (tens of millions of developers)
- As Java is close to [C++](#) and [C#](#), it makes it easy for programmers to switch to Java or vice versa

The Java language was designed with the following properties:

- **Platform independent:** Java programs use the Java virtual machine as abstraction and do not access the operating system directly. This makes Java programs highly portable. A Java program (which is standard-compliant and follows certain rules) can run unmodified on all supported platforms, e.g., Windows or Linux.
- **Object-orientated programming language:** Except the primitive data types, all elements in Java are objects.
- **Strongly-typed programming language:** Java is strongly-typed, e.g., the types of the used variables must be pre-defined and conversion to other objects is relatively strict, e.g., must be done in most cases by the programmer.
- **Interpreted and compiled language:** Java source code is transferred into the bytecode format which does not depend on the target platform. These bytecode instructions will be interpreted by the Java Virtual machine (JVM). The JVM contains a so called Hotspot-Compiler which translates performance critical bytecode instructions into native code instructions.
- **Automatic memory management:** Java manages the memory allocation and de-allocation for creating new objects. The program does not have direct access to the memory. The so-called garbage collector automatically deletes objects to which no active pointer exists.
- **High Performance:** Minimum Hardware requirement

- **Simple**

Java is considered as one of simple language because it does not have complex features like Operator overloading, [Multiple inheritance](#), pointers and Explicit memory allocation.

- **Robust Language**

Robust means reliable. Java programming language is developed in a way that puts a lot of emphasis on early checking for possible errors, that's why java compiler is able to detect errors that are not easy to detect in other programming languages. The main features of java that makes it robust are garbage collection, Exception Handling and memory allocation.

- **Secure**

We don't have pointers and we cannot access out of bound arrays (you get `ArrayIndexOutOfBoundsException` if you try to do so) in java. That's why several security flaws like stack corruption or buffer overflow is impossible to exploit in Java.

- **Multithreading**

Java supports [multithreading](#). Multithreading is a Java feature that allows concurrent execution of two or more parts of a program for maximum utilization of CPU.

- **Portable**

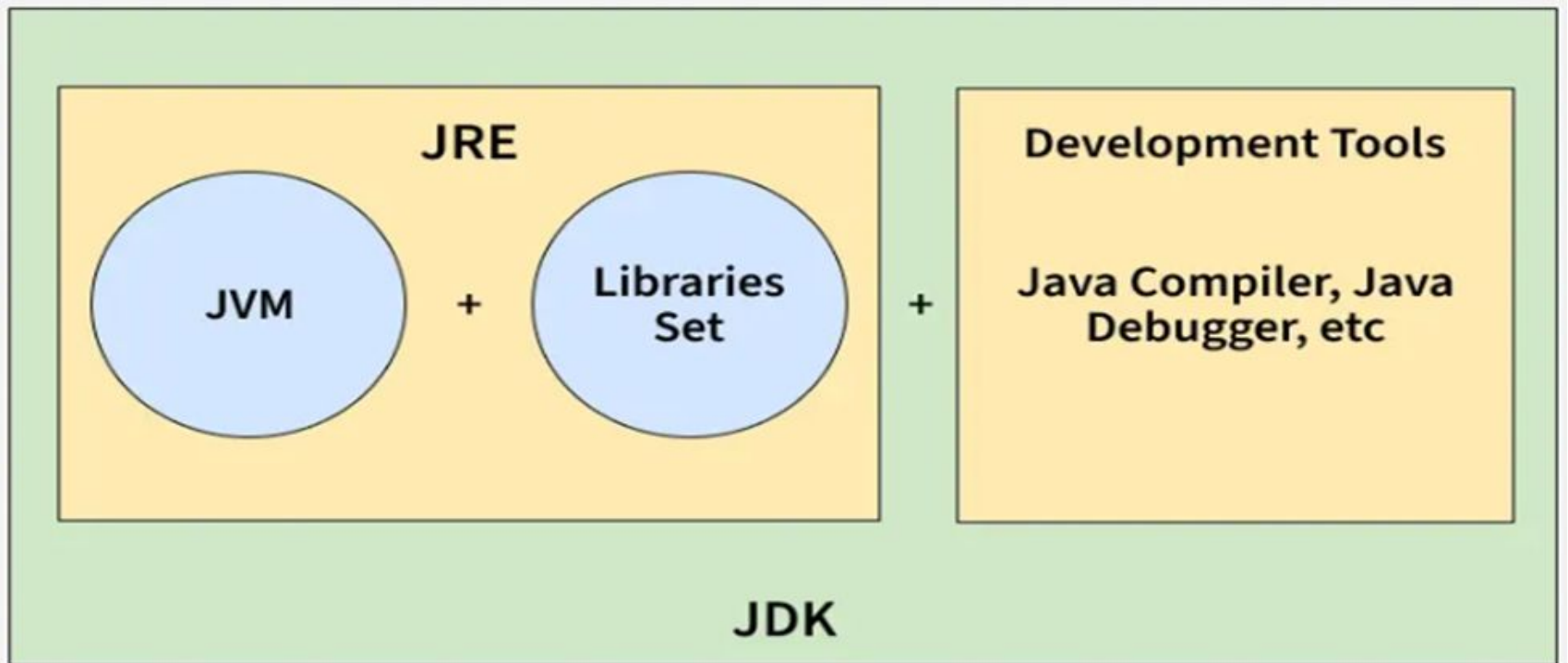
As discussed above, java code that is written on one machine can run on another machine. The platform independent byte code can be carried to any platform for execution that makes java code portable.

Java Development Kit (JDK)

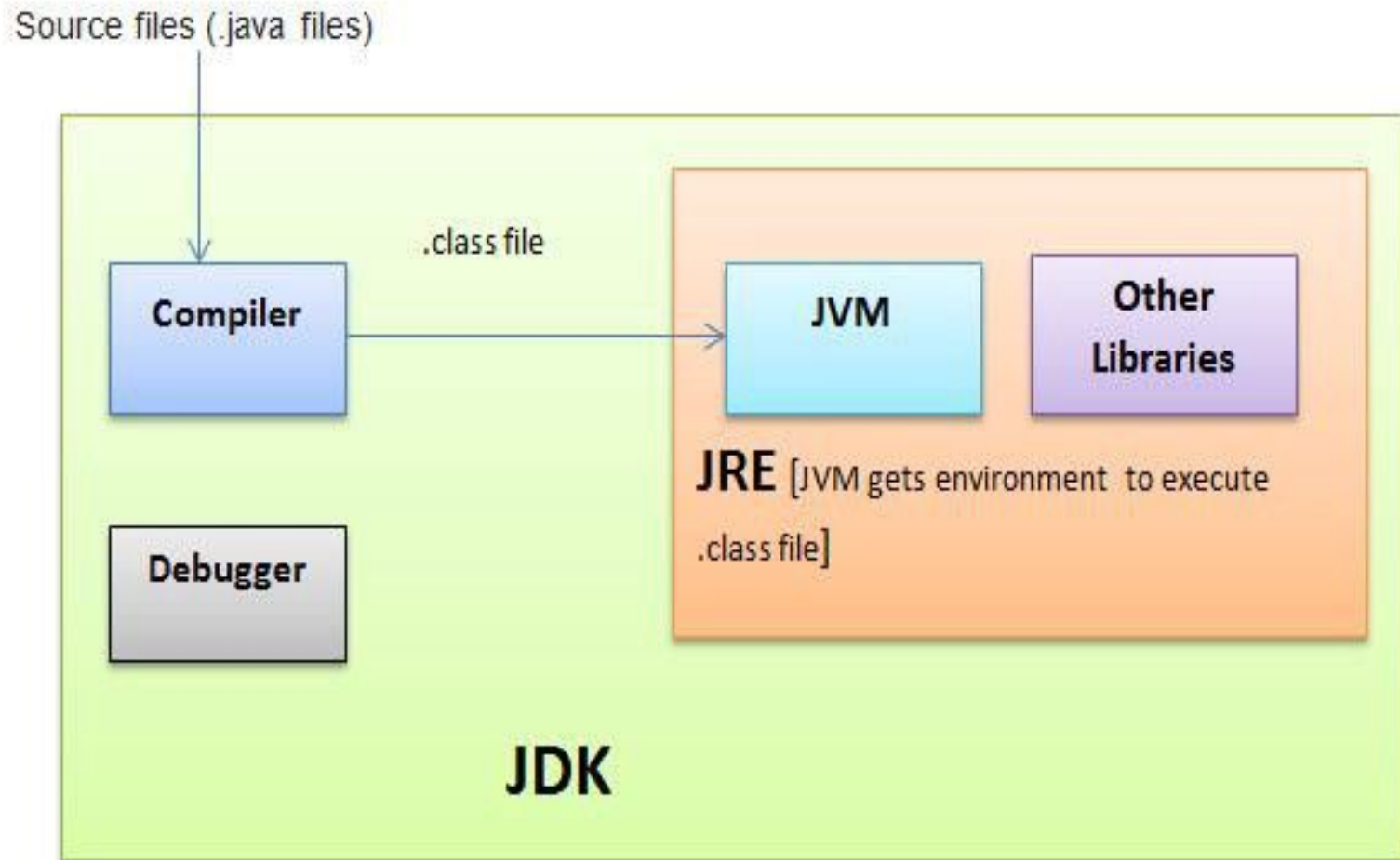
- Java Development Kit (JDK) is a cross-platform software development kit that provides tools and libraries needed to build Java-based applications and applets.
- It works together with the JVM (Java Virtual Machine) and JRE (Java Runtime Environment) as part of the core Java setup.
- The JDK includes everything required for Java development — compilers, debuggers, and other essential tools.
- Beginners often get confused between JRE and JDK.
- If you only want to run Java programs, installing JRE is enough.
- But if you want to develop Java applications, you'll need the JDK, which includes the JRE and additional development tools.

- The Java Development Kit (JDK) is a software development environment used for developing Java applications and applets.
- It includes the Java Runtime Environment (JRE), an interpreter/loader (Java), a compiler (javac), an archiver (jar), a documentation generator (Javadoc) and other tools needed in Java development.

JDK Architecture



JVM & JRE



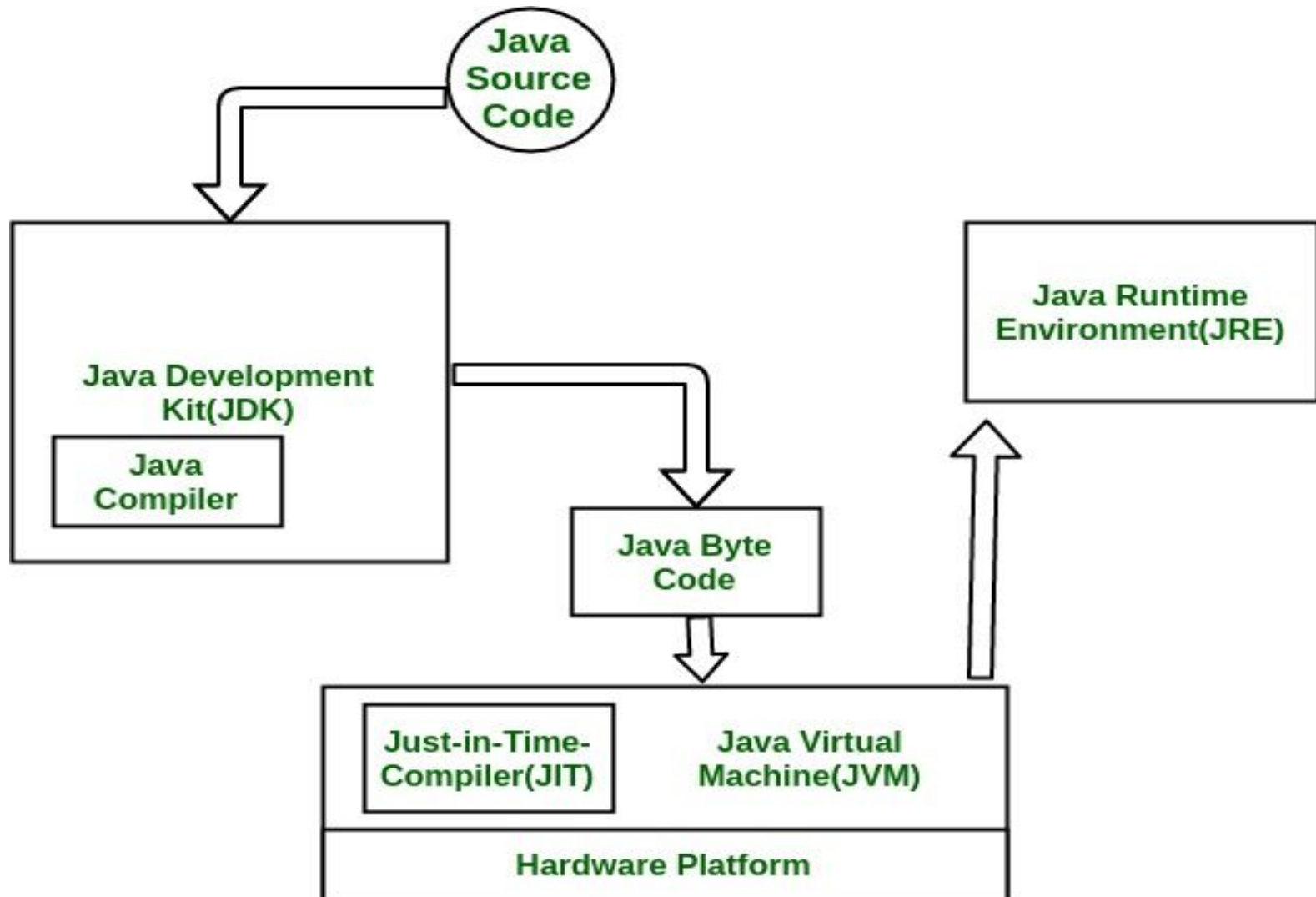
- **JAVA RUNTIME ENVIRONMENT**

- JRE stands for “Java Runtime Environment” and may also be written as “Java RTE.” The Java Runtime Environment provides the minimum requirements for executing a Java application; it consists of the Java Virtual Machine (JVM), core classes, and supporting files.

- **JVM – Interpreter**

- **Java Virtual machine(JVM)** is a very important part of both JDK and JRE because it is contained or inbuilt in both.
- Whatever Java program you run using JRE or JDK goes into JVM and JVM is responsible for **executing the java program line by line** hence it is also known as **interpreter**.

JVM Architecture



JDK Components:

- **appletviewer:** This tool is used to run and debug Java applets without a web browser.
- **java:** The loader for Java applications. This tool is an interpreter and can interpret the class files generated by the javac compiler.
- **javac:** It specifies the Java compiler, which converts source code into Java bytecode.
- **javadoc:** The documentation generator, which automatically generates documentation from source code comments.
- **javap:** the class file disassembler.
- **JConsole:** Java Monitoring and Management Console.
- **jdb:** Java debugger, which helps us to find errors in our programs.

Application Programming Interface

An API is a collection of prewritten packages, classes, and interfaces with their respective methods, fields and constructors.

Similar to a user interface, which facilitates interaction between humans and computers, an API serves as a software program interface facilitating interaction.

- **Language Support package:** A collection of classes and method required for implementing basic features of java.
- **Utility Package:** A collection of classes to provide utility functions such as date and time function.
- **Input/output Package:** A Collection of classes required for input/output manipulation.
- **Networking package:** A collection of classes for communicating with other computers via internet.
- **AWT package:** The Abstract Window Tool kit package contain classes that implement platform-independent graphical user interface.
- **Applet package:** This include a set of classes that allow us to create Java applet.

- **Object-Oriented Programming**

Object-oriented programming (OOP) is a programming paradigm based upon objects (having both data and methods) that aims to incorporate the advantages of modularity and reusability. Objects, which are usually instances of classes, are used to interact with one another to design applications and computer programs. The data of an object can be accessed only by the methods associated with that object. However methods of one object can accessed the methods of other objects.

The important features of object-oriented programming are –

- Bottom-up approach in program design
- Programs organized around objects, grouped in classes
- Focus on data with methods to operate upon object's data
- Interaction between objects through functions
- Reusability of design through creation of new classes by adding features to existing classes

- **Top-Down Approach**

The basic idea in top-down approach is to break a complex algorithm or a problem into smaller segments called modules, this process is also called as *modularization*. The modules are further decomposed until there is no space left for breaking the modules without hampering the originality. The uniqueness of the problem must be retained and preserved. The decomposition of the modules is restricted after achieving a certain level of modularity. The top-down way of solving a program is step-by-step process of breaking down the problem into chunks for organising and solving the sole problem. **The C- programming language uses the top-down approach of solving a problem in which the flow of control is in the downward direction.**

Bottom-Up Approach

As the name suggests, this method of solving a problem works exactly opposite of how the top-down approach works. In this approach we start working from the most basic level of problem solving and moving up in conjugation of several parts of the solution to achieve required results. The most fundamental units, modules and sub-modules are designed and solved individually, these units are then integrated together to get a more concrete base to problem solving.

Top-down Approach

A top-down approach is essentially the breaking down of a program to gain insight into its compositional small program (or module) in a reverse engineering fashion.

Structure / procedure oriented programming languages like C programming language follows top-down approach.

A top-down approach begins with high level design and ends with low level design or development.

In top-down approach, main function is written first and all sub functions are called from main function thus, sub-functions are written based on the requirement

Bottom-up Approach

A bottom-up approach is the piecing together of module (or small program) to give rise to more complex program, thus making the original modules of the emergent program.

Object oriented programming languages like C++ and JAVA programming language follows bottom-up approach.

A bottom-up approach begins with low level design or development and ends with high level design.

In bottom-up approach, code is developed from modules and then these modules are integrated with main function

Object Oriented Concept:

- **Encapsulation and Data Hiding**

Encapsulation

Encapsulation is the process of binding both attributes and methods together within a class. Through encapsulation, the internal details of a class can be hidden from outside. It permits the elements of the class to be accessed from outside only through the interface provided by the class.

Data Hiding

Typically, a class is designed such that its data (attributes) can be accessed only by its class methods and insulated from direct outside access. This process of insulating an object's data is called data hiding or information hiding.

- **Message Passing**

Any application requires a number of objects interacting in a harmonious manner. Objects in a system may communicate with each other using message passing. Suppose a system has two objects: obj1 and obj2. The object obj1 sends a message to object obj2, if obj1 wants obj2 to execute one of its methods.

The features of message passing are –

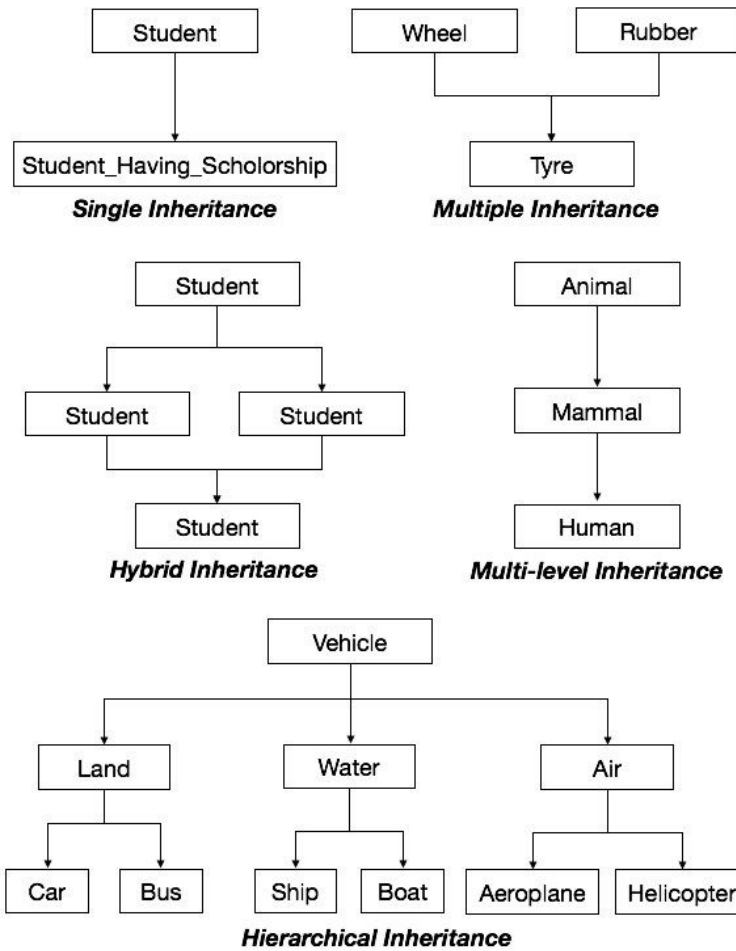
- Message passing between two objects is generally unidirectional.
- Message passing enables all interactions between objects.
- Message passing essentially involves invoking class methods.
- Objects in different processes can be involved in message passing.

- **Inheritance**

Inheritance is the mechanism that permits new classes to be created out of existing classes by extending and refining its capabilities. The existing classes are called the base classes/parent classes/super-classes, and the new classes are called the derived classes/child classes/subclasses. The subclass can inherit or derive the attributes and methods of the super-class(es) provided that the super-class allows so. Besides, the subclass may add its own attributes and methods and may modify any of the super-class methods. Inheritance defines an “is – a” relationship.

Types of Inheritance

- **Single Inheritance** – A subclass derives from a single super-class.
- **Multiple Inheritance** – A subclass derives from more than one super-classes.
- **Multilevel Inheritance** – A subclass derives from a super-class which in turn is derived from another class and so on.
- **Hierarchical Inheritance** – A class has a number of subclasses each of which may have subsequent subclasses, continuing for a number of levels, so as to form a tree structure.
- **Hybrid Inheritance** – A combination of multiple and multilevel inheritance so as to form a lattice structure.



- **Polymorphism**

Polymorphism is originally a Greek word that means the ability to take multiple forms. In object-oriented paradigm, polymorphism implies using operations in different ways, depending upon the instance they are operating upon. Polymorphism allows objects with different internal structures to have a common external interface. Polymorphism is particularly effective while implementing inheritance. For example consider the operation of addition. For two numbers, the operation will generate a sum. If the operands are strings, then the operation would produce a third string by concatenation.

Java "Hello, World!" Program

// Your First Program

```
class HelloWorld
```

```
{
```

```
public static void main(String[] args)
```

```
{
```

```
System.out.println("Hello, World!");
```

```
}}}
```

If you have copied the exact code, you need to save the file name as HelloWorld.java. It's because the name of the class and filename should match in Java.

- In Java, any line starting with// is a comment
- In Java, every application begins with a class definition

```
public      static      void      main(String[]      args)      {      ...      }
```

This is the main method. Every application in Java must contain the main method. The Java compiler starts executing the code from the main method.

Parameters used in First Java Program

Let's see what is the meaning of class, public, static, void, main, String[], System.out.println().

- **class** keyword is used to declare a class in java.
- **public** keyword is an access modifier which represents visibility. It means it is visible to all.
- **static** is a keyword. If we declare any method as static, it is known as the static method. The core advantage of the static method is that there is no need to create an object to invoke the static method. The main method is executed by the JVM, so it doesn't require to create an object to invoke the main method. So it saves memory.
- **void** is the return type of the method. It means it doesn't return any value.
- **main** represents the starting point of the program.
- **String[] args** is used for command line argument.
- **System.out.println()** is used to print statement. Here, System is a class, out is the object of PrintStream class, println() is the method of PrintStream class.

How to set path in Java

- The path is required to be set for using tools such as javac, java, etc.
- If you are saving the Java source file inside the JDK/bin directory, the path is not required to be set because all the tools will be available in the current directory.
- However, if you have your Java file outside the JDK/bin folder, it is necessary to set the path of JDK.

How to set the Path of JDK in Windows

- To set the path of JDK, you need to follow the following steps:
- Open the command prompt
- Copy the path of the JDK/bin directory
- Write in command prompt: set path=copied_path

For Example:

- set path=C:\Program Files\Java\jdk1.6.0_23\bin

The **command-line arguments** in Java allow us to pass arguments during the execution of the program.

Example

The following program displays all of the command-line arguments that it is called with -

```
public class CommandLine
{
    public static void main(String args[])
    {
        for(int i = 0; i<args.length; i++)
        {
            System.out.println("args[" + i + "]: " + args[i]);
        }
    }
}
```

Try executing this program as shown here –
\$java CommandLine this is a command line 200
-100

Output

This will produce the following result -

args[0]: this

args[1]: is

args[2]: a

args[3]: command

args[4]: line

args[5]: 200

args[6]: -100

Scope of Variables in Java:

Java variables are actually classified into three kinds –

- Instance variable
- Class variable
- Local variable

Instance variable and class variable are declared inside a class. Instance variable are created when the objects are instantiated and therefore they are associated with the objects. They take different values for each object.

On the other hand, class variables are global to a class and belong to the entire set of objects that class creates. Only one memory location is created for each class variable.

Variable declared and used inside methods are called local variables. They are called so because they are not available for use outside the method definition.

- **Type casting:** the process of converting one data type to another is called casting. Casting is often necessary when a method return a type different from the one we require.

e.g. malloc()

The process of assigning a smaller type to a larger one is known as **widening or promotion** and that of assigning a larger type to a smaller one is known as **narrowing**. Note that narrowing may result in loss of information.

- **Increment and decrement operator:**

m=5;

y=++m; in this case y and m would be 6.

m=5;

y=m++; in this case y would be 5 and m would be 6.

- **Instanceof operator:**

The instanceof is an object reference operator and return true if the object on the left-hand side is an instance of the class given on the right-hand side. This operator allows us to determine whether the object belongs to a particular class or not.

Example: **person instanceof student**

Is true if the object person belong to the class student; otherwise it is false

Java Abstract Classes and Methods

- Data **abstraction** is the process of hiding certain details and showing only essential information to the user. Abstraction can be achieved with either **abstract classes** or [interfaces](#). The **abstract** keyword is a non-access modifier, used for classes and methods:
- **Abstract class:** is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class).
- **Abstract method:** can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (inherited from).
- An abstract class can have both abstract and regular methods:

```
abstract class Animal
{
public abstract void animalSound();
public void sleep()
{
System.out.println("Zzz");
}
}
```

From the example above, it is not possible to create an object of the Animal class:

Animal myObj = new Animal(); // will generate an error

To access the abstract class, it must be inherited from another class.

```
// Abstract class
abstract class Animal
{
    public abstract void animalSound(); // Abstract method (does not have a body)
    public void sleep() // Regular method
    {
        System.out.println("Zzz");
    }
}

class Pig extends Animal // Subclass (inherit from Animal)
{
    public void animalSound()
    {
        System.out.println("The pig says: wee wee");
    }
}

class MyMainClass
{
    public static void main(String[] args)
    {
        Pig myPig = new Pig(); // Create a Pig object
        myPig.animalSound();
        myPig.sleep();
    } }
```

Why And When To Use Abstract Classes and Methods?

- To achieve security - hide certain details and only show the important details of an object.
- This allows us to manage complexity by omitting or hiding details with a simpler, higher-level idea.

Points to Remember:

- An abstract class must be declared with an abstract keyword.
- It can have abstract and non-abstract methods.
- It cannot be instantiated.
- It can have [constructors](#) and static methods also.
- To implement features of an abstract class, we inherit subclasses from it and create objects of the subclass.
- It can have final methods which will force the subclass not to change the body of the method.
- If subclass does not implement all the abstract methods of the superclass, then subclass may be declare abstract.

Why can't we create the object of an abstract class?

- Because these classes are incomplete, they have abstract methods that have no body so if java allows you to create object of this class then if someone calls the abstract method using that object then What would happen? There would be no actual implementation of the method to invoke. Also because an object is concrete. An abstract class is like a template, so you have to extend it and build on it before you can use it.

```
class Rectangle
{
int length,width;
void getData(int x,int y)
{
length=x; width=y;
}
int rectArea()
{   int area=length*width;
return(area);
} }
class RectArea()
{
public static void main(String args[])
{
int area1, area2;
Rectangle rect1=new Rectangle();
Rectangle rect2=new Rectangle();
rect1.length=15;
rect1.width=10;
area1=rect1.length*rect1.width;
rect2.getData(20,12);
area2=rect2.rectArea();
System.out.println("Area1 =" + area1);
System.out.println("Area2 =" + area2);
}
}
```

- **Java Constructors**

A constructor in Java is a **special method** that is used to initialize objects. The constructor is called when an object of a class is created. It can be used to set initial values for object attributes:

- Note that the constructor name must **match the class name**, and it cannot have a **return type** (like void).
- Also note that the constructor is called when the object is created.
- All classes have constructors by default: if you do not create a class constructor yourself, Java creates one for you. However, then you are not able to set initial values for object attributes.

Types of Java constructors

There are two types of constructors in Java:

- Default constructor (no-arg constructor)
- Parameterized constructor

```
class Rectangle
{
int length,width;
Rectangle (int x, int y)
{
length=x;
width=y;
}
int rectArea()
{
return(length * width);
} }
class RectArea()
{
public static void main(String args[])
{

Rectangle rect1=new Rectangle(15,10);
int area1=rect1.rectArea();
System.out.println("Area1 =" + area1);
}
}
```

Static Members:

- In Java, static members are those which belongs to the class and you can access these members without instantiating the class.
- The static keyword can be used with methods, fields, classes (inner/nested), blocks.
- **Static Methods** – You can create a static method by using the keyword *static*. Static methods can access only static fields, methods. To access static methods there is no need to instantiate the class, you can do it just using the class name.

Advantages of static variable

- It makes your program **memory efficient** (i.e., it saves memory).

```
class MathOperation
{
    static float mul(float x, float y)
    {
        return(x * y);
    }
    static float divide(float x, float y)
    {
        return(x /y);
    }
}
class MathApplication
{
    public static void main(String args[])
    {
        float a=MathOperation.mul(4.0,5.0);
        float b=MathOperation.divide(a,2.0);
        System.out.println("b=" +b);
    }
}
```

Method Overloading and Method Overriding in Java:

Method Overloading is a **Compile time polymorphism**. In method overloading, more than one method shares the same method name with different signature in the class. In method overloading, return type can or can not be same, but we must have to change the parameter because in java, we can not achieve the method overloading by changing only the return type of the method.

```
class MethodOverloadingEx
{
    static int add(int a, int b)
    {
        return a+b;
    }
    static int add(int a, int b, int c)
    {
        return a+b+c;
    }
    public static void main(String args[])
    {
        System.out.println(add(4, 6));
        System.out.println(add(4, 6, 7));
    }
}
```

- Method Overriding is a **Run time polymorphism**. In method overriding, derived class provides the specific implementation of the method that is already provided by the base class or parent class. In method overriding, return type must be same.

```
class Animal
{
void eat()
{
System.out.println("eating.");
}
}
class Dog extends Animal
{
void eat()
{
System.out.println("Dog is eating.");
}
}
class MethodOverridingEx
{
public static void main(String args[])
{
Dog d1=new Dog();
Animal a1=new Animal();
d1.eat();
a1.eat();
}
}
```


- Here, we can see that a method eat() has overridden in the derived class name **Dog** that is already provided by the base class name **Animal**. When we create the instance of class Dog and call the eat() method, we see that only derived class eat() method run instead of base class method eat() and When we create the instance of class Animal and call the eat() method, we see that only base class eat() method run instead of derived class method eat().
- So, its clear that in method overriding, method is bound to the instances on the run time which is decided by the **JVM**. That's why it is called **Run time polymorphism**

- Manipulation of strings

```
class StringManipulation
{
    public static void main(String args[])
    {
        StringBuffer str=new StringBuffer("object language");

        System.out.println("original string:" +str);

        System.out.println("length of string :" + str.length());
        //accessing character in a string

        for(int i=0; i<str.length(); i++)
        {
            int p=i+1;
            System.out.println("character at position" + p + "is" + str.charAt(i));
        }
    }
}
```

```
String astring=new String(str.toString());  
int pos=astring.indexOf(" language");  
str.insert(7,"oriented");
```

```
System.out.println("modified string" + str);  
    str.setCharAt(6,'-');  
System.out.println("string now" + string);
```

```
str.append("improve security");
```

```
System.out.println("Appended string" + str);
```

```
}
```

```
}
```